Vector

Lecture 18

Based on Slides of Dr. Norazah Yusof

Vector

Size of an array is fixed for the duration of the execution program.
Vector is a more flexible data structure which allows its size to be changed.

- A vector class is part of the java.util package
- It can dynamically shrink or grow as needed
- A data element can be inserted into or removed from any location of a vector
- The elements of a vector must be objects (cannot be primitive types - if you need to store a primitive type, you need to use appropriate wrapper classes).
- A Vector is basically the same as an ArrayList, but Vector methods are synchronized for thread safety.

From the Library: java.util.Vector

- **Regular arrays are limited to their initial size. They cannot grow or shrink. (*static*)**
- **The java.util.Vector class implements an array of objects that can grow as needed. *(dynamic)***
  - **the size of a vector does not have to be declared - it grows and shrinks as needed**
- **There are methods to add , retrieve, insert in the middle of the vector, or remove elements**

Vectors - API documentation (partial listing)

```
public class Vector extends Object implements Cloneable, Serializable
{
 // constructors
  public Vector (int initialCapacity);
  public Vector();
 // Instance methods for different functions
 public final void addElement (Object obj); - insert element at next free location
 public final void insertElement (Object obj, int index); - insert element at certain location
 public final int  capacity();   - return capacity of vector
 public final Object elementAt(int index);- return object stored at the position 'index'
 public final Object firstElement(); - return first element (object at vector position 0)
 public final int  indexOf(Object elem); - return index of the given element in vector
 public final int  lastIndexOf(Object elem); - return last index of the given element in vector
 public final Object lastElement(); - return last element (object at position size-1)
 public final synchronized boolean removeElement(Object obj); - removes specified object
 public final synchronized void removeAllElements(); - removes all objects
 public final synchronized void removeElementAt(); - removes object at certain index
 public final int size(); - return number of objects stored in the vector
 public final void trimToSize(); - reduce capacity of the vector to the number of elements
}
```

Allows specification of initial capacity of vector

- **Use vector to store a collection of data that contains texts (i.e. faculty names).**
- **Import statement:**

```
import java.util.*;
```

- **Declare a `Vector` object of faculty:**

```
Vector faculty = new Vector();
```

  - **A `Vector` of size 0 is created.**

Example 1 of Vector

- **Add new element to a data collection:**

```
faculty.addElement ("FSKSM");
faculty.addElement ("FS");
faculty.addElement ("FKE");
faculty.addElement ("FAB");
```

  - **The `Vector` will contain three elements in location 0, 1, 2 and 3.**
- **Display the content of vector:**

```
System.out.println(faculty);
```

- **Display the size of the vector:**

```
System.out.println(faculty.size());
```

Example 1 of Vector

- **To insert new element at the third element of the vector:**

```
faculty.insertElementAt ("FPPSM", 2);
```

  - **`"FPPSM"` is stored in between `"FS"`, and `"FKE"`.**
- **Display the content of vector:**

```
System.out.println(faculty);
```

- **Display the size of the vector:**

```
System.out.println(faculty.size());
```

- **To remove element with content `"FKE"` in the vector:**

```
faculty.removeElement ("FKE");
```

Example 1 of Vector

- **To remove element at the third element of the vector:**

```
faculty.removeElementAt (2);
```

  - `"FS"` **will be deleted.**
- **To display the content of vector at the specified element:**

```
System.out.println (faculty.elementAt(1));
```

  - `"FPPSM"` **will be displayed.**
- **To change the value of specific element of a vector**

```
faculty.setElementAt("FKSG",2);
```

  - **The content of element three will be replaced by** `"FKSG"`.

Vector (cont.): Example 2

```
import java.util.Vector;
class Perisian_Komputer  {
        public static void main(String[]arg)  {
                Vector perisian = new Vector();
                String nama = new String("Visual Image");
                Integer siri = new Integer(1235);
                Double harga = new Double(2134.5);
                String pengeluar = new String("Oracle");
                Character test = new Character('a');
                Float code = new Float(12.34f);
```

Vector (cont.): Example 2(cont.)

```
perisian.addElement(nama);
perisian.addElement(siri);
perisian.addElement(harga);
perisian.addElement(pengeluar);
perisian.addElement(test);
perisian.addElement(code);

System.out.println("Kandungan vektor:");
System.out.println(perisian);

}  }
```

Wrapper Classes for primitive types,

- **Example of use: store in Vectors (where primitive types cannot be stored).**

```
double x = 13.5;
Vector numbers  = new Vector();
numbers.addElement(new Double(x));
double y = ((Double) numbers.elementAt(0)).doubleValue();
```

Assume vector v contains a collection of objects, some of which are type Double. The statements below store some objects in vector v. The loop adds only the numbers wrapped in the type Double objects and displays their sum (50.5).

```
Vector v = new Vector();
v.addElement("Jamil");              // first element is a string
v.addElement(new Double(15.0));   // second element is type Double
v.addElement(new Double(35.5));   // third element is type Double
v.addElement(new Integer(100));   // fourth element is type Integer

double sum = 0.0;
for (int i = 0; i < v.size(); i++) {
   Object temp = v.elementAt(i);
   if (temp instanceof Double)
       sum + = ((Double) temp).doubleValue();
}
System.out.println("Sum of elements in vector v is " + sum);
```

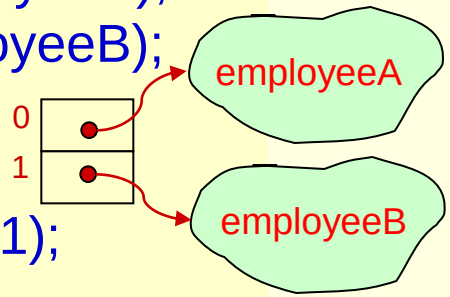
stores new Integer *object in* v.

true if temp *is type* Double

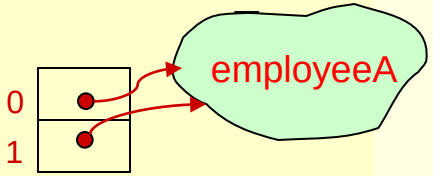
Vectors Example

Use the **Vector** class to store a collection of employee:

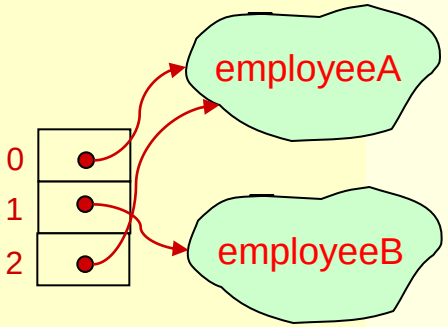
- import stmt: import java.util.*;
- data declaration: Vector employees = new Vector();
- add existing objects: employees.addElement(employeeA);
  employees.addElement(employeeB);



- access 2nd element:
  anEmployee = (Employee) employees.elementAt(1);
- change an element:
  employees.setElementAt(employeeA, 1);



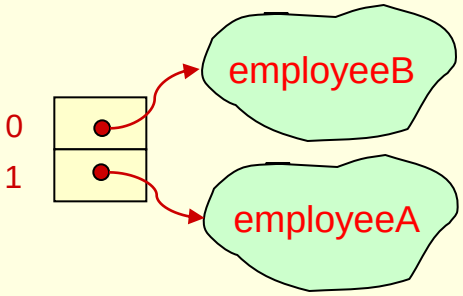
- insert an element:
  employees.insertElementAt(employeeB, 1);



Vectors example, cont'd.

remove an element:

```
employees.removeElementAt(0);
```



Methods for Vector class

Method call	Effect
Vector()	Constructs an empty vector (type of elements does not need to be specified)
Employees.addElement(Emp1)	Appends object Emp1 to vector employees and increments its size
(Employee)employees.elementAt(0)	Returns the object at index 0 and converts it to type Employee
Employees.size()	Returns the size of vector employees
employees.setElementAt(Emp2, 1)	Changes the object at index 1 to Emp2
employees.insertElementAt(anEmp, 1)	Insert object anEmp at index 1, shifting the elements that follow up one position. Increment vector size.
employees.removeElementAt(1)	Remove the element at index 1, shifting the elements that follow down one position. Decrement vector size.
employees.toString()	Forms a string enclosed in square brackets. The objects in employees are separated by commas.

Exercises

- **Do the Exercises in page 180 – 186 in the lab book.**